

BORDER OPTICS — POORI RESEARCH SAMJHO

**PDF 14 of 22 — Robustness and Advanced Stats: Wild Cluster
Bootstrap**

wild_cluster_bootstrap.py

Sakshi ke liye, Sakshi ki apni research par. PDF 12 mein humne `did_model.py` seekha tha, aur ek chhota-sa warning bhi note kiya tha — ki DiD ki cluster-robust standard errors sirf 14 districts par based hain, jo ek genuinely kam sankhya hai. Ye PDF wahi warning ko seedha face karta hai — aur is poore project ka sabse computationally-heaviest, sabse rigorous check banata hai: 2 ki power 14 barabar 16,384 possible sign-combinations ka poora, exact enumeration.

Aur ek interesting twist bhi hai: is file ko likhne ke liye `statsmodels` library available hi nahi thi, isliye poori OLS regression aur cluster-robust SE formula haath se, `numpy` mein, scratch se likhi gayi — aur trust karne se pehle, usse `did_model.py` ke already-published numbers ke against verify kiya gaya.

Shuru Karne Se Pehle — Wahi Purana Sawaal Hai, Lekin Is Baar Poochne Ka Tareeka Alag Hai

PDF 12 mein `did_model.py` ne H4 test kiya tha — kya treated villages, control villages ke muqable mein zyada badle? Jawaab har jagah null tha, aur standard errors district-level clustering ke saath compute hui thi (`cov_type="cluster"`, `groups=district`). Us waqt humne ek caveat note kiya tha jise abhi tak sirf mention kiya gaya tha, poori tarah address nahi kiya gaya tha: cluster-robust standard errors ka poora statistical theory 'bahut saare clusters' hone ka assumption karta hai — usually 30-50 ya usse zyada. Is study mein sirf 14 districts hain. Iska matlab ye nahi ki `did_model.py` ka result galat hai — matlab sirf itna hai ki usi result ko ek doosre, is chhote sample-size ke liye specially design kiye gaye method se bhi check kar lena chahiye, taaki confidence ho ki 14-cluster ki kamzori kahin result ko artificially significant ya artificially null toh nahi bana rahi.

Wild cluster bootstrap (Cameron, Gelbach aur Miller, 2008) exactly isi problem ke liye banaya gaya method hai. Iska core idea simple hai, chahe implementation thodi involved ho: agar hume nahi pata ki normal-theory formula 14 clusters ke saath bharosemand hai ya nahi, toh formula par bharosa karne ke bajaye, hum seedha data se hi dikhaa sakte hain ki 'agar sach mein koi effect nahi hota, toh humari t-statistic kaisi dikhti' — aur phir asli, real t-statistic ko us banaayi hui tasveer ke saath compare kar sakte hain. Is chapter mein hum yehi dekhenge — line by line, formula se leke code tak.

File Ka Poora Naam Aur Uski History

`wild_cluster_bootstrap.py` Development Log ki Entry 32 mein pehli baar likhi gayi thi — is study ki apni 'research complete hai kya?' checklist mein jo paanch items abhi bhi khule the, unmein se ek yehi tha: paper ki apni threats-to-validity table mein ye line thi ki 'a wild-cluster bootstrap was not run.' Entry 32 ne ye gap band kiya. Fir Entry 37 mein, jab control-group ki district-verification wali ek alag, unrelated bug fix hui (732 se 721 villages), toh har wo analysis jo control-group data par depend karti thi — including ye file — dobara chalayi gayi. Is PDF ka Final State section dono baar ke numbers dikhaayega: Entry 32 ka original 732-village run, aur Entry 37 ka current, corrected 721-village run.

Pehle Samjho: '14 Clusters Bahut Kam Hain' Ka Matlab Kya Hai

PDF 12 mein humne clustering ka concept seekha tha — ki ek hi district ke andar ke villages ek-doesre se independent nahi hain (agar us district mein kuch alag ho raha hai — jaise koi local administrative policy, ya ek jaisa terrain — toh us district ke saare villages usse mildte-julte tarike se affect honge). Isliye district-level clustering use ki gayi thi, taaki standard errors is 'ek-doesre se judi hui' nature ko account kar sakein.

Lekin cluster-robust standard error ka formula khud ek statistical approximation hai, aur har approximation ki apni sharten hoti hain jinke andar hi wo achi tarah kaam karta hai. Is formula ka core assumption ye hai ki clusters ki sankhya (yahan district ki sankhya, G) 'kaafi badi' honi chahiye — jitni zyada districts hongi, utna hi behtar ye formula apne aap ko 'average out' kar payega, aur uski standard error utni hi bharosemand hogi. Statistics literature mein ek rough, commonly-cited guideline hai ki formula ke reliable hone ke liye kam se kam 30-50 clusters chahiye. 14 districts is guideline se kaafi neeche hain.

EXAMPLE — Ek Chhoti Class Ka Example

Socho ek teacher 50 students ke ek bade school mein average marks nikaal rahi hai — agar wo 50 students ko 10 alag classrooms mein baant kar, har classroom ka apna average nikaale, fir un 10 averages ka overall pattern dekhe, toh 10 classroom-averages se ek stable, trustworthy overall picture ban sakti hai, kyunki 10 apne aap mein ek reasonable sankhya hai. Lekin agar sirf 2 classrooms hote, aur teacher un 2 classroom-averages se hi poore school ka pattern samajhne ki koshish karti, toh us andaaze mein bahut zyada uncertainty hogi — sirf isliye ki '2' ek bahut chhota sample hai groups ke level par, chahe har classroom mein khud kitne bhi students kyun na hon. 14 districts ka case bhi kuch aisa hi hai — total 1944 observations (village-year rows) hain, jo apne aap mein achi khaasi sankhya hai, lekin wo saari observations sirf 14 groups mein baati hui hain, aur cluster-robust SE formula in 14 groups ko hi apna 'sample size' maanta hai, individual observations ko nahi.

Iska solution ye nahi hai ki clustering hata di jaaye — clustering hataana toh problem ko aur bigaad dega (villages ko wapas 'independent' maan lena, jo galat hai). Solution ye hai ki ek aisa test use kiya jaaye jo 14-cluster jaisi chhoti sankhya ke saath bhi mathematically sahi rahe — aur wild cluster bootstrap exactly wahi test hai.

Wild Cluster Bootstrap Ka Core Idea — Formula Ke Bina Bhi Samjho

Poori technique ka core idea ek single sawaal hai: 'agar sach mein treated aur control villages mein koi farak hi na hota (yaani DiD ka asli effect zero hota), toh sirf random statistical noise ki wajah se, mujhe kitni baar itni badi ya usse bhi badi t-statistic mil sakti thi?' Agar jawaab hai 'bahut kam baar' — matlab asli result genuinely unusual hai, aur ye ek real effect ka signal ho sakta hai. Agar jawaab hai 'kaafi baar' — matlab asli result utna khaas nahi hai jitna wo normal-theory formula se dikh raha tha, kyunki sirf random noise se hi aise numbers routine tarike se aa jaate hain.

Ye sawaal poochne ke liye, technique ek 'no effect' duniya banaati hai (ise restricted model kehte hain — jismein DiD term hi hataa diya jaata hai, jaise ye maan liya gaya ho ki uska asli effect zero hai), us duniya ke fitted values aur residuals (errors) nikaalti hai, fir un residuals ko district-district karke sign flip karti hai (poore ek district ka sign ek saath palta jaata hai — kabhi +1, kabhi -1 — taaki district ke andar ki correlation wali structure preserve rahe), ek nayi fake dataset banati hai, us par poora model (DiD term ke saath) dobara fit karti hai, aur uski t-statistic record karti hai. Ye poora process baar-baar (is file mein, saari 16,384 possible combinations ke liye, ek bhi chhoote bina) dohraya jaata hai, taaki ek 'agar koi effect na hota toh t-statistic kaisi dikhti' wali poori distribution ban jaaye. Aakhir mein, asli t-statistic ko is distribution ke saath compare karke p-value nikaali jaati hai.

Ab Line-By-Line — wild_cluster_bootstrap.py

Docstring Aur Imports

```
"""
Wild cluster bootstrap for the H4 DiD results (14 clusters is too few for
normal asymptotics). Did OLS + cluster-robust SE by hand since statsmodels
wouldn't install here - checked it matches did_model.py's numbers first.
Full enumeration of all 2**14=16384 sign flips (exact, not Monte Carlo,
since that's small enough to just do directly).
"""

import itertools
import json

import numpy as np
import pandas as pd
```

Purpose: Docstring khud is file ki poori kahani ek jagah bata deta hai: 14 clusters 'too few' hain normal asymptotics (yaani normal-theory cluster-robust SE formula) ke liye — isliye ye alag, robust method chahiye. statsmodels install nahi ho payi thi is sandbox mein jahan ye code likha gaya (PyPI package-registry access policy se block tha), isliye OLS aur cluster-robust SE khud, haath se likhi gayi — lekin trust karne se pehle did_model.py ke already-published numbers se match check ki gayi. Full enumeration ka matlab hai '2 ki power 14 barabar 16384' saari possible sign-combinations, ek-ek karke, poori ki poori — koi random sampling ya approximation nahi. Syntax: import itertools — Python ki ek built-in library jo combinations/permutations jaisi cheezein generate karne ke liye bani hai; is file mein pehli baar use ho rahi hai, aage combos generate karne ke liye. import json, numpy as np, pandas as pd — teeno is poore series mein pehle bhi dekhe ja chuke hain (JSON save karna, numerical arrays, dataframes).

Paths

```
PANEL_PATHS = {  
    "full_year": "data/processed/border_optics_did_panel_fullyear.csv",  
    "summer": "data/processed/border_optics_did_panel_summer.csv",  
}  
OUT_JSON = "outputs/wild_cluster_bootstrap_results.json"
```

Purpose: PANEL_PATHS wahi do panel files hain jo PDF 12 mein did_model.py ke build_panel() function ne banayi thi — treated aur control villages ka wide-to-long reshape kiya hua, before/after wala data, jismein treatment, post, aur did_term columns already ban chuke hain. Ye file un panels ko dobara nahi banaati, seedha reuse karti hai — isliye wild-cluster bootstrap aur did_model.py exactly ek hi underlying dataset par kaam karte hain. Syntax: Dictionary literal, PDF 12 se bilkul same pattern — string key, file-path string value.

manual_ols() — OLS Regression, Ek Formula Se Seedha

```
def manual_ols(y, X):  
    XtX_inv = np.linalg.inv(X.T @ X)  
    beta = XtX_inv @ X.T @ y  
    resid = y - X @ beta  
    fitted = X @ beta  
    return beta, resid, fitted, XtX_inv
```

Purpose: Ye function OLS (ordinary least squares) regression ko uske sabse fundamental matrix-algebra formula se seedha compute karta hai — bilkul wahi cheez jo statsmodels ka smf.ols(...).fit() internally karta hai, bas yahan wo internal kaam khud, khule taur pe likha gaya hai. β (regression coefficients) = $(X'X)^{-1} X'y$ — ye har statistics course mein OLS ka 'closed-form' formula sikhaya jaata hai. resid = actual y minus fitted (model ne jo predict kiya uska farak asli value se) — yahi wo 'errors' hain jo cluster_robust_se() aage use karega. fitted = model ka prediction, $X @ \beta$ se seedha nikalta hai. Syntax: $\text{np.linalg.inv}(M)$ — numpy ka matrix-inversion function, ek matrix M ka mathematical 'inverse' nikaalta hai (jaise numbers mein $1/x$ hota hai, waise matrices mein M^{-1} hota hai). $X.T$ — matrix transpose, rows aur columns palat dena. $@$ operator — Python mein ye matrix multiplication ke liye reserved hai (normal $*$ element-wise multiplication karta, $@$ proper matrix-multiply karta hai — dono alag cheezein hain). Function ek saath 4 values return karta hai, jaise multiyear_trend.py mein stats.linregress() 5 values return karta tha.

EXAMPLE — $(X'X)^{-1} X'y$ Ka Matlab Kya Hai, Bina Formula Ke

Ek seedhi rekha (line of best fit) sochiye jaisi PDF 13 mein stats.linregress() ke saath dekhi thi — wahan sirf ek x-variable (saal) tha. Yahan X poore ek 'design matrix' hai jismein constant, treatment, post, did_term, aur district dummy columns — sab ek saath, kayi columns — shaamil hain. $(X'X)^{-1} X'y$ wahi 'best-fit line/plane' dhoondhne wala formula hai, bas ab ek line ke bajaye ek multi-dimensional 'best fit' hai jo ek

saath saari X columns ko account karta hai. Numpy jaisi library ke liye ye ek seedha, well-known matrix operation hai — statsmodels bhi bilkul yehi formula (thoda zyada optimized tarike se) internally chalata hai jab aap `smf.ols(...).fit()` likhte hain.

cluster_robust_se() — Sandwich Formula, Khud Se Likhi Gayi

```
def cluster_robust_se(X, resid, clusters, XtX_inv):
    groups = pd.unique(clusters)
    G = len(groups)
    N, K = X.shape
    B = np.zeros((K, K))
    for g in groups:
        idx = clusters == g
        Xg = X[idx]
        ug = resid[idx]
        score = Xg.T @ ug
        B += np.outer(score, score)
    corr = (G / (G - 1)) * ((N - 1) / (N - K))
    V = XtX_inv @ (corr * B) @ XtX_inv
    return np.sqrt(np.diag(V))
```

Purpose: Ye function bilkul wahi cheez implement karta hai jo `did_model.py` mein `cov_type="cluster", cov_kwds={"groups": ...}` likhne se statsmodels apne andar chhupa ke kar deta hai — bas yahan har step khula, dikhne wala hai, kyunki statsmodels available hi nahi thi. Har district (group) ke liye, us district ki saari rows ka 'score' ($Xg.T @ ug$ — X aur residual ka combination) nikaala jaata hai, fir usse khud ke saath outer-product karke B mein jod diya jaata hai — ye is baat ka hisaab rakhta hai ki ek hi district ke andar ki errors ek-doesre se kitni related hain. `corr` ek small-sample correction factor hai — jab clusters ki sankhya (G) chhoti ho, tab raw formula thodi 'neeche ki taraf biased' hoti hai, isliye ye factor use adjust karta hai. Aakhir mein V (poori variance-covariance matrix) 'sandwich' formula se banti hai — do baar `XtX_inv` (jaise 'bread', bread-slices) ke beech mein B (jaise 'meat', filling) rakh kar — isliye is tarah ke SE formula ko literature mein 'sandwich estimator' bhi kaha jaata hai. **Syntax:** `pd.unique(arr)` — ek array ki saari unique values nikaalta hai, jaise `sorted(df["col"].unique())` PDF 12 mein dekha tha, bas yahan sorting nahi hai. `X.shape` — numpy array ka (rows, columns) tuple, ek saath do variables (N, K) mein unpack ho raha hai. `np.zeros((K,K))` — sabhi zero values wali ek K-by-K matrix banata hai, ek khaali 'accumulator' jismein loop ke andar values jodi jaayengi. `clusters == g` — boolean mask, us district ki rows True, baaki sab False. `np.outer(a,a)` — ek vector ka 'outer product' khud ke saath, ek poori matrix banata hai (contrast: dot product ek single number deta hai, outer product ek poori matrix). `np.diag(V)` — matrix V ki diagonal values nikaalta hai; `np.sqrt()` unka square-root leta hai — standard errors hamesha variance ka square-root hoti hain.

build_design() — District Dummy Variables, Khud Bana Kar

```
def build_design(df, outcome):
    valid = df.dropna(subset=[outcome]).copy()
    y = valid[outcome].values.astype(float)
    district_dummies = pd.get_dummies(valid["district"], drop_first=True, prefix="district")
    X_full = pd.concat([
        pd.Series(1.0, index=valid.index, name="const"),
        valid[["treatment", "post", "did_term"]],
        district_dummies,
    ], axis=1).astype(float)
    cols = list(X_full.columns)
    X_restricted = X_full.drop(columns=["did_term"]).values
    clusters = valid["district"].values
    return y, X_full.values, cols, X_restricted, clusters
```

Purpose: `did_model.py` mein `C(district)` likhne se `statsmodels` apne aap district ko 'fixed effects' dummy-columns mein badal deta tha, bina hume kuch dikhaaye. Yahan `statsmodels` ka formula-syntax available nahi hai, isliye wahi kaam khud, explicitly kiya jaata hai — `pd.get_dummies()` se district ko multiple 0/1 columns mein todha jaata hai (ek column har district ke liye — 'is ye row Chamoli district ki hai? 1 ya 0', waise hi baaki districts ke liye). `drop_first=True` ek zaroori detail hai — agar saari 14 districts ke liye dummy column bana di jaaye, toh unka jod hamesha bilkul 1 aayega, jo constant column ke saath ek mathematical conflict (perfect multicollinearity) create karta hai; ek district ko 'baseline' (jise koi dummy column nahi milta) chhod dena isse theek kar deta hai. `X_restricted` wahi `X_full` hai lekin `did_term` column hataa kar — yahi wo 'no DiD effect' wali duniya hai jismein hum residuals nikaalenge (agla function). Syntax: `pd.get_dummies(col, drop_first=True, prefix="district")` — pandas ka built-in one-hot-encoding function, ek categorical column ko kayi 0/1 columns mein badalta hai, prefix se naam deta hai (jaise `district_Chamoli`). `pd.concat([...], axis=1)` — kayi columns ko side-by-side jod kar ek dataframe banata hai (`axis=1` matlab columns ki tarah jodna, rows ki tarah nahi — jaise `did_model.py` mein `axis=0` se rows jodi thi PDF 12 mein, yahan `axis=1` se columns). `pd.Series(1.0, index=..., name="const")` — ek poora column banata hai jismein har row ki value 1.0 hai — ye regression ka intercept/constant term hai. `list(df.columns)` — column-naam ek Python list mein. `df.drop(columns=["x"])` — ek specific column hataana. `.values` — dataframe/series ko plain numpy array mein badalna (`manual_ols()` ko numpy array chahiye, dataframe nahi).

wild_cluster_bootstrap() — Asli Test, Real Fit Se Shuru

```
def wild_cluster_bootstrap(outcome, window):
    df = pd.read_csv(PANEL_PATHS[window])
    y, X_full, cols, X_restricted, clusters = build_design(df, outcome)
    did_idx = cols.index("did_term")

    beta_real, resid_real, _, XtX_inv_real = manual_ols(y, X_full)
    se_real = cluster_robust_se(X_full, resid_real, clusters, XtX_inv_real)
```

```

real_coef = beta_real[did_idx]
real_se = se_real[did_idx]
real_t = real_coef / real_se

```

Purpose: `did_idx` dhoondh leta hai ki `X_full` ke columns mein `did_term` kaunse position (index number) par hai, taaki aage har baar beta aur SE arrays mein se sahi value nikaali ja sake. Fir asli, real data par poora model fit hota hai — `manual_ols()` se coefficients aur residuals, `cluster_robust_se()` se standard errors. `real_coef`, `real_se`, aur unse `real_t` (coefficient bhaag standard error, jo ek t-statistic ban jaata hai) — yahi wo teen numbers hain jo, agar sab kuch sahi likha gaya hai, toh `did_model.py` ke already-published DiD coefficient aur cluster-robust SE se match hone chahiye (verification jo docstring mein mention hui thi). Syntax: `cols.index("did_term")` — Python list ka `.index()` method, ek value ki position (0 se shuru hoti) dhoondh deta hai. Function-call se ek saath 4 values return hokar 4 variables mein unpack ho rahi hain — bilkul wahi pattern `manual_ols()` define karte waqt dekha tha.

```

beta_res, resid_res, fitted_res, _ = manual_ols(y, X_restricted)

unique_districts = sorted(pd.unique(clusters))
n_clusters = len(unique_districts)
district_idx = {d: i for i, d in enumerate(unique_districts)}
row_cluster_idx = np.array([district_idx[d] for d in clusters])

```

Purpose: Ab restricted model fit hota hai — `X_restricted` use karke, yaani `did_term` column ke bina, jaise maan liya gaya ho ki DiD ka asli effect zero hai. Isse milne wale `fitted_res` (predictions) aur `resid_res` (residuals) hi aage 'fake data' banaane ke kaam aayenge. Fir har district ko ek chhota integer number (0 se 13 tak) diya jaata hai — `district_idx` dictionary se — aur `row_cluster_idx` mein har observation (row) ke liye uske apne district ka wahi integer store hota hai. Ye setup hai taaki aage, jab har district ke liye ek sign decide hoga, hum turant jaan sakein ki kaunsi row kaunsa sign paayegi. Syntax: `sorted(pd.unique(arr))` — unique districts nikaal kar unhe alphabetically sort karna. `{d: i for i, d in enumerate(list)}` — dictionary comprehension, `enumerate()` se har item ko uski position (i) ke saath jodkar, ek dictionary bana di jaati hai jismein district-naam key hai aur uska integer-index value hai. `[district_idx[d] for d in clusters]` — list comprehension, har row ke district-naam ko uske integer-index se badalna, phir `np.array()` se पूरी list ko numpy array mein badal dena.

Sign-Combination Loop — Poore 16,384 Ka Full Enumeration

```

n_combos = 2 ** n_clusters
print(f"\n--- Wild cluster bootstrap, {outcome.upper()}, window={window} ---")
print(f"n_obs={len(y)} n_clusters={n_clusters} full enumeration = {n_combos} sign combinations")
print(f"Real fit: did_term={real_coef:+.5f} cluster-robust SE={real_se:.5f} t={real_t:+.3f}")

```

```

boot_t = []
for combo in itertools.product([-1, 1], repeat=n_clusters):
    signs = np.array(combo)[row_cluster_idx]
    y_star = fitted_res + signs * resid_res
    beta_b, resid_b, _, XtXinv_b = manual_ols(y_star, X_full)
    se_b = cluster_robust_se(X_full, resid_b, clusters, XtXinv_b)[did_idx]
    if se_b <= 0 or not np.isfinite(se_b):
        continue
    boot_t.append(beta_b[did_idx] / se_b)
boot_t = np.array(boot_t)

```

Purpose: $n_combos = 2^{14} = 16384$ — 14 districts hain, aur har district ke liye do hi possibilities hain (+1 ya -1), isliye total possibilities 2 baar-baar khud se multiply, 14 baar. `itertools.product([-1,1], repeat=14)` exactly yehi saari 16,384 combinations, ek-ek karke, generate karta hai — koi bhi chhote bina. Har combo (ek 14-length tuple, jaise (+1,-1,+1,+1,...)) ke liye: signs us combo ko `row_cluster_idx` se 'expand' karta hai — har district ka apna assigned sign, us district ki saari rows tak fail jaata hai. `y_star` ek nayi, fake outcome-variable banata hai — restricted model ke predictions (jinmein koi DiD effect nahi hai) plus sign-flipped residuals (jo real-world jaisa 'noise' add karte hain, lekin district ke andar wahi sign, taaki district ki internal correlation preserve rahe). Is fake data par poora model (`did_term` ke saath) dobara fit hota hai, uski t-statistic nikaal kar `boot_t` list mein jodi jaati hai. Ye poora process 16,384 baar hota hai. Syntax: $2^{**}14$ — ** Python ka 'power' operator (PDF 13 mein $r^{**}2$ mein dekha tha). `itertools.product(list, repeat=n)` — Cartesian product, matlab list ke saath khud ka combination, n baar — `[-1,1]` ke saath `repeat=2` se 4 combos ((-1,-1),(-1,1),(1,-1),(1,1)) milte, `repeat=14` se 16384. `for combo in ...:` — loop, har baar combo ek tuple hai. `np.array(combo)[row_cluster_idx]` — 'fancy indexing': combo ek chhota (14-length) array hai; `row_cluster_idx` ek bada (1944-length) array hai jismein sirf 0-13 ke numbers hain — jab combo ko `row_cluster_idx` se index kiya jaata hai, har position par `combo[us number]` nikal jaata hai, isliye asal mein 14 district-level signs expand hokar 1944 row-level signs ban jaate hain. `list.append(x)` — list mein item jodna, loop ke andar. `if condition: continue` — agar condition sahi hai, is iteration ko chhod kar seedha agli iteration par jaana (ek defensive/safety check, taaki koi pathological/broken case `boot_t` ko corrupt na kare).

EXAMPLE — Ek Chhote, 2-District Example Se Poori Cheez Samjho

Agar sirf 2 districts hote (14 ki jagah), toh `itertools.product([-1,1], repeat=2)` exactly 4 combinations deta: (-1,-1), (-1,1), (1,-1), (1,1) — matlab 'dono districts negative,' 'pehla negative doosra positive,' 'pehla positive doosra negative,' 'dono positive.' Har combination ke liye, poore us district ki saari rows ek saath wahi sign paatin — District A ki 100 rows sab -1, District B ki 80 rows sab +1, wagairah. Yahi tareeka 14 districts tak scale hota hai, bas ab 4 ki jagah 16,384 combinations hain. Ye 'poori tarah enumerate karna' isliye possible hai kyunki 16,384 numpy ke liye ek chhota

number hai — compute karne mein sirf seconds lagte hain, isliye 'sirf kuch random sample lena' (Monte Carlo) ki zaroorat hi nahi padi.

Randomization Inference (PDF 13) Se Farak — Full Enumeration vs Monte Carlo

PDF 13 mein `h3_border_proximity_robustness.py` ka `randomization_inference()` function bhi ek permutation-based p-value nikaalta tha — lekin usne 2000 random shuffles kiye the (`N_PERMUTATIONS=2000`, `RNG_SEED=42` ke saath), poori possible shuffles ki sankhya nahi. Wahan aisa isliye kiya gaya kyunki 251 villages ko shuffle karne ki total possible ways ki sankhya astronomically badi hai — usse poora enumerate karna practically impossible hai, isliye ek bada, random sample (2000 shuffles) le kar us bade space ko approximate kiya gaya tha (ise 'Monte Carlo' approach kehte hain). Yahan, district-level sign-flips ke liye total possibilities sirf 16,384 hain — ek chhota, poora manageable number — isliye poora enumeration hi kiya ja sakta hai, koi approximation ki zaroorat nahi. Full enumeration ka fayda ye hai ki jawaab bilkul exact hota hai, kisi random-sampling-noise ka koi risk nahi — jabki Monte Carlo approach mein, alag seed ya kam draws lene se jawaab thoda idhar-udhar ho sakta tha (isi liye PDF 13 mein `RNG_SEED` fix ki gayi thi, reproducibility ke liye). Dono methods ek hi family (permutation/resampling-based inference) ke hain, bas is baar total-space chhota hone ki wajah se poora, exact version istemal ho saka.

p-value Nikaalna, Aur main()

```
p_wcb = np.mean(np.abs(boot_t) >= np.abs(real_t))

print(f"Wild-cluster-bootstrap p-value (full enumeration, {len(boot_t)}/{n_combos} valid
draws) = {p_wcb:.5f}")
print(f" bootstrap t distribution: mean={boot_t.mean():+.3f} SD={boot_t.std():.3f} "
      f"[{np.percentile(boot_t, 2.5):+.3f}, {np.percentile(boot_t, 97.5):+.3f}]")

return {
    "outcome": outcome, "window": window, "n_obs": len(y), "n_clusters": n_clusters,
    "real_coef": float(real_coef), "real_cluster_se": float(real_se), "real_t":
float(real_t),
    "n_bootstrap_draws": int(len(boot_t)), "n_combinations_total": n_combos,
    "wild_cluster_bootstrap_p": float(p_wcb),
    "boot_t_mean": float(boot_t.mean()), "boot_t_sd": float(boot_t.std()),
}
```

Purpose: `p_wcb` yehi is poore file ka final jawaab hai: saari 16,384 'agar koi asli effect na hota' scenarios mein se, kitna fraction (0 se 1 ke beech) itni extreme (ya usse zyada extreme) t-statistic degate hain jitni real data mein mili? Agar ye fraction bada hai (jaisa is file mein har jagah mila — 0.35 se 0.68 ke beech), matlab real result koi khaas/rare cheez nahi hai — sirf random noise se hi routine tarike se itni badi

t-statistics milti rehti hain, isliye koi real DiD effect hone ka saboot nahi milta. `main()` function bilkul PDF 12 ke `did_model.py` jaisa hi loop chalata hai — do windows, do outcomes, chaaron combinations ke liye `wild_cluster_bootstrap()` call karta hai, aur results ko ek JSON file mein save karta hai. Syntax: `np.mean(condition_array)` — jab ek boolean array (True/False values) ko `np.mean()` diya jaata hai, wo True ko 1 aur False ko 0 treat karke average nikaalta hai — yaani seedha 'kitna fraction True hai' bata deta hai, ek elegant, ek-line ka tareeka. `np.abs(x)` — absolute value (negative sign hataana), taaki 'kitni extreme' compare karte waqt sirf magnitude dekhi jaaye, direction nahi. `>=` operator do arrays ke beech element-wise comparison karta hai, ek naya boolean array deta hai. `f"{x:+.5f}"` — f-string formatting jismein + sign hamesha dikhaya jaata hai (positive numbers ke aage bhi +), aur 5 decimal places tak round kiya jaata hai — is poore series mein pehle bhi dekha gaya pattern, bas yahan + flag naya hai. `np.percentile(arr, 2.5)` — ek array ka wo value jiske neech 2.5% data aata hai, distribution ki 'tails' dekhne ke liye use hota hai.

Entry 32 Se Entry 37 Tak — Ek History Jo Kisi Result Ko Palti Nahi, Sirf Sharpen Karti Hai

EXAMPLE — Entry 32 — 'statsmodels Nahi Mili,' Isliye Sab Kuch Haath Se

Jab `wild_cluster_bootstrap.py` pehli baar likhi gayi (Entry 32), tab pata chala ki jis sandbox environment mein ye kaam ho raha tha, wahan `statsmodels` library ko naya install nahi kiya ja sakta tha (PyPI package-registry access policy-block tha). Iska matlab tha ki `did_model.py` jaisi seedha, one-line `smf.ols(...).fit(cov_type="cluster",...)` wali approach yahan available nahi thi — poori OLS regression aur cluster-robust sandwich SE formula khud, numpy se, likhni padi (yahi wo `manual_ols()` aur `cluster_robust_se()` functions jo abhi humne padhe). Lekin ek naye, haath-se-likhe implementation par seedha bharosa karna risky hota — agar formula mein kahin ek chhoti si bhi galti hoti, toh saare aage ke numbers galat hote. Isliye trust karne se pehle, ek zaroori step uthaaya gaya: is manual implementation se, `did_model.py` ke real DiD coefficient aur cluster-robust standard errors ko reproduce karke dekha gaya — aur ye kam se kam 10 significant figures tak match hui. Sirf tabhi is naye code ko wild-cluster bootstrap jaisi bhaari computation ke liye trust kiya gaya. Ye ek acha, generalizable lesson hai — jab bhi koi naya, khud likha hua implementation kisi established library ka kaam dohraata hai, use trust karne se pehle kisi jaani-maani, already-verified value ke against test kar lena chahiye.

EXAMPLE — Entry 37 — District-Verification Fix, Aur Ek Chhota Sa Number-Shift (Reversal Nahi)

Is poore project ki history mein, jab bhi koi data-quality bug fix hui hai — 154-village incomplete sample (PDF 11), control-group contamination (PDF 12) — toh pattern hamesha same raha: significant result, fix ke baad, null ban gaya. Entry 37 ka

district-verification fix (jo control-list ko 732 se 721 villages tak sudhaarta hai — Tawang aur Pithoragarh ab genuine Nominatim boundary polygons se verify hote hain, sirf bbox-overlap se nahi) bhi har control-group-dependent analysis ko dobara chalaane ki maang karta hai, including `wild_cluster_bootstrap.py`. Lekin is baar result palta nahi — sirf thoda shift hua, kyunki ye result pehle se hi null tha, dono baar. Entry 32 ka original, 732-village-list run: full-year NDBI $p=0.349$, full-year lights $p=0.324$, summer NDBI $p=0.495$, summer lights $p=0.465$ — sab null. Entry 37 ka current, 721-village-list rerun: full-year NDBI $p=0.35394$, full-year lights $p=0.50488$, summer NDBI $p=0.58533$, summer lights $p=0.67957$ — phir se sab null, bas exact decimal-values thodi badal gayi (kyunki underlying data ke 251 treated villages toh same hain, lekin unke against compare kiye jaane wale 721 control villages ka set thoda badal gaya — 11 villages ki jagah geodesically-verified villages ne le li). Ye is baat ka ek achha reminder hai ki data-quality fixes har analysis ko touch karte hain — chahe wo analysis pehle se hi null ho aur us fix se 'kuch bada' na badalne wala ho, phir bhi use dobara chalaana zaroori hai, sirf isliye maan lena ki 'ye toh waise bhi null hai' galat approach hoga.

Final State — Wild Cluster Bootstrap Ka Current, Stress-Tested Verdict

Ye `outputs/wild_cluster_bootstrap_results.json` se seedha liya gaya, current (721-village-list, Entry 37 ke baad ka), chaaron-combination ka poora result. Har combination ke liye poori 16,384/16,384 sign-combinations valid nikli — koi bhi combination skip nahi hui (`se_b<=0` wala defensive check kabhi trigger nahi hua):

Window / Outcome	DiD Coefficient	Cluster-Robust SE	Real t	WCB p-value
Full-year NDBI	+0.00668	0.00633	+1.056	0.35394
Full-year Lights	+0.03460	0.04614	+0.750	0.50488
Summer NDBI	-0.00275	0.00439	-0.627	0.58533
Summer Lights	+0.01798	0.03891	+0.462	0.67957

Chaaron combinations ka DiD coefficient aur cluster-robust SE, aur `real_t` bhi, exactly wahi hain jo `did_model.py` (PDF 12) mein already-published the — ye confirm karta hai ki `manual_ols()` aur `cluster_robust_se()` ne statsmodels ke kaam ko sahi tarah reproduce kiya. Naya, extra number hai WCB p-value — aur chaaron cases mein ye normal cluster-robust p-values (jo `did_model.py` mein hi report hui thi, 0.29 se 0.64 ke beech) se agree karti hai, thodi zyada hokar (0.35 se 0.68 ke beech) — matlab ye zyada rigorous, 14-clusters-ke-liye-khaas-design-kiya-gaya test bhi wahi conclusion deta hai

jo simple cluster-robust formula pehle se de rahi thi: H4 ka koi bhi result significant nahi hai, kisi bhi window ya outcome mein, chahe test kitna bhi strict kyun na ho.

Is poore file ka sabse important takeaway ye hai ki ye null-result ko sirf 'maan lena' nahi hai — ye us null-result ko is study ka sabse chhota (14), sabse zyada doubt-worthy cluster-count ke against, sabse zyada rigorous possible tareeke se — full enumeration, koi approximation nahi — test karke confirm karna hai. Agar ye result normal-theory formula ke saath significant dikhta aur wild-cluster bootstrap ke saath null ho jaata, tabhi ye chapter genuinely chinta-janak hota. Aisa nahi hua — dono methods ek hi kahani sunaate hain.

Is PDF mein humne dekha: cluster-robust standard errors ka formula 'bahut saare clusters' hone ka assumption karta hai, aur is study ke sirf 14 districts hain — isliye ek alag, chhote-cluster-ke-liye-design-kiya-gaya test chahiye tha. `wild_cluster_bootstrap.py` ne OLS aur cluster-robust SE ko haath se (numpy mein) implement kiya, kyunki statsmodels available nahi thi — aur trust karne se pehle `did_model.py` ke numbers se verify kiya (10 significant figures tak match). Restricted model (no DiD effect) ke residuals district-district karke sign-flip karke, poori $2^{14}=16384$ combinations ka full (exact, Monte-Carlo nahi) enumeration kiya gaya, ek 'agar koi effect na hota toh t-statistic kaisi dikhti' wali null-distribution banaane ke liye. Chaaron H4 combinations null rahi ($p=0.354$ se 0.680), consistent with normal-theory cluster-robust p-values — Entry 32 ke 732-village original run se Entry 37 ke 721-village corrected run tak, numbers thoda shift hue lekin conclusion palti nahi, is project ki history mein aisa hone wala pehla case.

Agla PDF (PDF 15) placebo tests dekhega — `pretreatment_placebo_test.py` aur `dw_pretreatment_placebo_test.py`, jo poochte hain: 'agar hum 2019 aur 2021 ke beech (jab VVP-I abhi shuru bhi nahi hui thi) wahi DiD test chalayein, toh kya waisa hi null milta hai jaisa 2021-2025 mein mila?' Jab bhi tayyar ho, bata dena.